Project CYRANO

Voice-Driven AI Companion

Version 1.0.0 | Production Ready | Google Cloud Platform

Gemini 2.5 Flash Native Audio | Cloud Run | Firestore | Secret Manager

Executive Summary

Mission Statement

Project CYRANO is a voice-driven AI companion that provides real-time conversational assistance through natural speech interaction, leveraging Google's Gemini 2.5 Flash Native Audio for low-latency multimodal processing.

Core Technology Stack

- Gemini 2.5 Flash Native Audio
- Google ADK (Agent Development Kit)
- Cloud Run (Serverless Deployment)
- Firestore (Session Persistence)
- Secret Manager (Credentials)

Service Metrics

- Port: 8080
- Session Timeout: 1800 seconds (30 minutes)
- Max Concurrent Requests: 10
- Cold Start Target: < 2 seconds

Deployment Target

Production deployment on Google Cloud Platform with automatic scaling and HTTPS termination.

System Architecture

Backend Stack

- Python 3.11+
- FastAPI Framework
- Uvicorn ASGI Server
- Google ADK Runtime
- Gemini Live API

Frontend Stack

- Vanilla JavaScript
- Web Audio API
- AudioWorklet Processor
- WebSocket Client
- Tailwind CSS (CDN)

Audio Specifications

- Input: 16kHz PCM
- Output: 24kHz PCM
- Chunk Size: 4096 samples
- Encoding: Base64

Database Architecture

```
# Firestore Schema

# Collection: cyrano_sessions
# Document ID: {session_id}
# Example: 550e8400-e29b-41d4-a716-446655440000
# Fields:
#   - session_id: string (UUID)
#   - user_id: string (default: "default_user")
#   - resumption_handle: string (Gemini session resumption token)
#   - last_updated: timestamp (UTC, for TTL cleanup)
#   - created_at: timestamp (UTC, for analytics)
#   - metadata: map (optional, extensible)
#     - client_version: string
#     - user_agent: string
#     - ip_address: string (hashed, optional)
#     - total_duration_seconds: number
#     - turn_count: number
#     - model_version: string
#
# Indexes:
#   - Primary: session_id (auto)
#   - Secondary: user_id + last_updated (composite)
#   - TTL: last_updated + 1800 seconds
#
# Operations:
#   - Create: When new session starts
#   - Read: Every 10 seconds during active session
#   - Update: Every 10 seconds during active session
#   - Delete: Manual cleanup or TTL expiration
#
# Example Document:
# {
# "session_id": "550e8400-e29b-41d4-a716-446655440000",
# "user_id": "default_user",
# "resumption_handle": "CAESJGNkMjFmZjAtMjAyMy0wMDAwLTAwMDAtMDAwMDAwMDAwMDAwMDAwMDAwMDA",
# "last_updated": 2025-12-31T14:30:45.123Z,
# "created_at": 2025-12-31T14:00:00.000Z,
# "metadata": {
# "client_version": "1.0.0",
# "user_agent": "Mozilla/5.0 (Chrome/120.0.0.0)",
# "ip_address": "hashed_abc123",
# "total_duration_seconds": 1800,
# "turn_count": 45,
# "model_version": "gemini-2.5-flash-native-audio-preview-12-2025"
# }
# }

# Firestore Security Rules
# rules_version = '2';
# service cloud.firestore {
# match /databases/{database}/documents {
# // Allow service account full access
# match /cyrano_sessions/{session_id} {
# allow read, write: if true;
# }
# }
# }

# Alternative: Cloud SQL (PostgreSQL)
# If Firestore is not suitable, use Cloud SQL
# Table: cyrano_sessions
# CREATE TABLE cyrano_sessions (
# session_id UUID PRIMARY KEY,
# user_id VARCHAR(255) DEFAULT 'default_user',
# resumption_handle TEXT,
# last_updated TIMESTAMP WITH TIME ZONE,
# created_at TIMESTAMP WITH TIME ZONE,
# metadata JSONB,
# );
# CREATE INDEX idx_user_id_last_updated ON cyrano_sessions(user_id, last_updated);
# CREATE INDEX idx_last_updated ON cyrano_sessions(last_updated);
# -- TTL cleanup:
# DELETE FROM cyrano_sessions WHERE last_updated < NOW() - INTERVAL '1800 seconds';

# Redis Alternative
# If low-latency required, use Redis
# Key: cyrano:session:{session_id}
# Value: JSON serialized session data
# TTL: 1800 seconds
# Commands:
# SET cyrano:session:550e8400... {"session_id":...} EX 1800
# GET cyrano:session:550e8400...
# DEL cyrano:session:550e8400...
# EXPIRE cyrano:session:550e8400... 1800

# Data Retention Policy
# Active sessions: Keep for 30 minutes (TTL)
# Inactive sessions: Auto-delete after 30 minutes
# Analytics: Export to BigQuery for long-term storage
# Backup: Daily snapshots to Cloud Storage (optional)

# Monitoring Queries
# Active sessions count:
# SELECT COUNT(*) FROM cyrano_sessions WHERE last_updated > NOW() - INTERVAL '60 seconds';
# Average session duration:
# SELECT AVG(metadata.total_duration_seconds) FROM cyrano_sessions WHERE created_at > NOW() - INTERVAL '1 hour';
# Total turns today:
# SELECT SUM(metadata.turn_count) FROM cyrano_sessions WHERE created_at > CURRENT_DATE;
# Stale sessions (should be cleaned):
# SELECT COUNT(*) FROM cyrano_sessions WHERE last_updated < NOW() - INTERVAL '1800 seconds';
```

CYRANO AI Agent System

```
# CYRANO AI Agent System
# Voice-Driven Companion with Gemini Live API
# Version: 1.0.0
# Model: gemini-2.5-flash-native-audio-preview-12-2025

# Core Identity
# You are CYRANO, a voice-driven AI companion.
# Your name stands for: Conversational Yielding Response Assistant Navigation Operator

# Primary Capabilities
# - Real-time voice conversation
# - Natural interruption handling
# - Context-aware assistance
# - Multilingual support
# - Emotion detection and response

# Conversation Flow
# 1. Listen actively to user speech
# 2. Detect when user needs assistance
# 3. Provide concise, relevant responses
# 4. Yield to interruptions immediately
# 5. Maintain context across turns

# Response Guidelines
# - Keep responses under 30 seconds when possible
# - Use natural, conversational language
# - Avoid technical jargon unless user uses it
# - Confirm understanding when ambiguous
# - Offer to elaborate only when asked

# Interruption Handling
# When user interrupts:
# 1. Immediately stop current response
# 2. Acknowledge briefly if appropriate ("Sorry, go ahead")
# 3. Listen to new input
# 4. Do not resume previous response unless asked

# Tone Adjustments
# Default: Friendly, helpful, professional
# Technical topics: Precise, detailed
# Casual conversation: Warm, relaxed
# Urgent requests: Direct, efficient

# Prohibited Behaviors
# - Do not pretend to be human when sincerely asked
# - Do not provide medical, legal, or financial advice
# - Do not store personal information
# - Do not interrupt user mid-sentence
# - Do not apologize excessively

# Error Handling
# If unclear: "I didn't catch that. Could you repeat?"
# If technical error: "Sorry, I'm experiencing a technical issue. Please try again."
# If out of scope: "That's outside my capabilities. I can help with [X, Y, Z]."

# Greeting
# Initial greeting when session starts:
# "Hello! I'm CYRANO, your voice assistant. How can I help you?"

# Farewell
# Ending conversation:
# "Goodbye! Feel free to come back anytime."

# Technical Integration
# Audio Input: 16kHz PCM, Base64 encoded
# Audio Output: 24kHz PCM, Base64 encoded
# Transcription: Enabled for both input and output
# Session Resumption: Supported via handle
# Context Window Compression: Enabled for long sessions

# Monitoring Metrics
# Track:
# - Response latency
# - Interruption frequency
# - Session duration
# - Turn count
# - Error rate
# - User satisfaction (implicit from continuation)

# Version History
# 1.0.0 - Initial production release
# - Gemini 2.5 Flash Native Audio integration
# - Bidirectional audio streaming
# - Session persistence
# - Interruption handling
# - Multilingual support
```

API Endpoints

```
# Endpoint Reference
# Base URL: https://cyrano-service-{hash}-uc.a.run.app
# Protocol: HTTP/1.1 and WebSocket
# Authentication: None (public endpoint)
# Content-Type: application/json
# Rate Limit: 10 concurrent connections
# Timeout: 300 seconds (WebSocket)
# Cold Start: ~2 seconds
# Warm Response: <100ms
# Health Check
# GET /health
# Response: 200 OK
# Content-Type: text/plain
# Body: OK
# Use Cases:
# - Load balancer health probe
# - Kubernetes liveness probe
# - Manual verification
# Root Information
# GET /
# Response: 200 OK
# Content-Type: application/json
# Body:
# {
# "service": "CYRANO",
# "description": "Voice-Driven AI Companion",
# "status": "operational",
# "endpoints": {
# "health": "/health",
# "websocket": "/ws/{session_id}"
# }
# }
# Use Cases:
# - Service discovery
# - Basic connectivity test
# - Documentation
# WebSocket Connection
# WS /ws/{session_id}
# Protocol: WebSocket (RFC 6455)
# Upgrade: HTTP/1.1 → WebSocket
# Parameters:
# session_id: UUID v4 (e.g., 550e8400-e29b-41d4-a716-446655440000)
# Client → Server Messages:
# 1. Audio Chunk
# {
# "type": "audio",
# "data": "<base64_encoded_pcm_16khz>"
# }
# Specifications:
# Sample Rate: 16kHz
# Channels: 1 (mono)
# Encoding: PCM 16-bit signed little-endian
# Chunk Size: 4096 samples (recommended)
# Frequency: Every 250ms (recommended)
# 2. Ping
# {
# "type": "ping"
# }
# Purpose: Keep connection alive
# Response: Pong
# Server → Client Messages:
# 1. Audio Response
# {
# "type": "audio",
# "data": "<base64_encoded_pcm_24khz>"
# }
# Specifications:
# Sample Rate: 24kHz
# Channels: 1 (mono)
# Encoding: PCM 16-bit signed little-endian
# Chunk Size: Variable
# Frequency: Real-time streaming
# 2. Transcription
# {
# "type": "transcription",
# "speaker": "user" | "assistant",
# "text": "transcribed text",
# "partial": true | false
# }
# Speaker: "user" = User speech, "assistant" = CYRANO speech
# Partial: true = Interim result, false = Final result
# 3. Interruption
# {
# "type": "interruption"
# }
# Purpose: Indicate user interrupted assistant
# Action: Stop playing current audio
# 4. Pong
# {
# "type": "pong"
# }
# Purpose: Response to ping
# Connection Lifecycle
# 1. Client connects to /ws/{session_id}
# 2. Server accepts WebSocket upgrade
# 3. Bidirectional streaming begins
# 4. Client sends audio chunks continuously
# 5. Server sends transcriptions and responses
# 6. Either side can send ping/pong
# 7. Interruptions handled in real-time
# 8. Connection closed on timeout or error
# Error Codes
# 1000 - Normal closure
# 1001 - Going away
# 1006 - Abnormal closure (no close frame)
# 1011 - Server error
# Session Persistence
# Storage: Firestore
# Collection: cyrano_sessions
# Document: {session_id}
# Fields:
# - session_id: string
# - user_id: string
# - resumption_handle: string
# - last_updated: timestamp
# - created_at: timestamp
# - metadata: map
# Operations:
# Create: When new session starts
# Read: Every 10 seconds during active session
# Update: Every 10 seconds during active session
# Delete: Manual cleanup or TTL expiration
# Resumption Flow
# 1. Client disconnects (network error, timeout, etc.)
# 2. Client reconnects with same session_id
# 3. Server loads resumption_handle from Firestore
# 4. Server passes handle to Gemini API
# 5. Gemini restores context
# 6. Conversation continues seamlessly
# Rate Limits
# Concurrent Connections: 10 per instance
# Total Connections: Unlimited (scale with instances)
# Audio Bandwidth: ~256 kbps per connection
# Monitoring
# Metrics:
# - Response latency
# - Interruption frequency
# - Session duration
# - Turn count
# - Error rate
# - Cold start frequency
# Logs:
# - Connection events
# - Errors and exceptions
# - Session lifecycle
# Alerts:
# - Error rate > 5%
# - Latency > 500ms
# - Memory > 80%
# Testing
# Health Check:
# curl https://cyrano-service-{hash}-uc.a.run.app/health
# Root Info:
# curl https://cyrano-service-{hash}-uc.a.run.app/
# WebSocket:
# wscat -c wss://cyrano-service-{hash}-uc.a.run.app/ws/550e8400-e29b-41d4-a716-446655440000
# Load Test:
# Max 10 concurrent connections
# Monitor memory and CPU
# Check for connection drops
# Security
# Current: Public endpoint, no authentication
# Recommendations:
# - Add API key authentication
# - Implement JWT tokens
# - Rate limit by IP address
# - Monitor for abuse
# Deployment
# Platform: Cloud Run
# Region: us-central1
# Scaling: Automatic (0-10 instances)
# Memory: 512MB
# CPU: 1 vCPU
# Timeout: 300 seconds
# Cost Estimate
# Idle: $0 (scale to zero)
# Active: ~$0.00002400 per second
# Example (1 hour): ~$0.086
# Documentation
# Interactive: https://cyrano-service-{hash}-uc.a.run.app/docs
# Schema: https://cyrano-service-{hash}-uc.a.run.app/openapi.json
# Support
# Issues: Check logs in Cloud Console
# Debug: Enable verbose logging
# Contact: Project maintainer
```

Google Cloud Deployment

```
# Google Cloud Deployment
# Project: gemini-cyrano
# Region: us-central1
# Service: cyrano-service
# Prerequisites
# 1. Google Cloud SDK installed
# 2. Authenticated: gcloud auth login
# 3. Project created: gcloud projects create gemini-cyrano
# 4. Billing enabled
# 5. Secrets configured in Secret Manager
# Quick Deploy
# ./deploy.sh
# Manual Deploy
# gcloud run deploy cyrano-service \
# --source . \
# --region us-central1 \
# --allow-unauthenticated \
# --memory 512Mi \
# --cpu 1 \
# --timeout 300 \
# --max-instances 10 \
# --set-secrets=GOOGLE_API_KEY=google-api-key:latest
# Verify Deployment
# gcloud run services list --region us-central1
# gcloud run services describe cyrano-service --region us-central1
# Logs
# gcloud run services logs read cyrano-service --region us-central1 --limit 50
# Rollback
# gcloud run services update-traffic cyrano-service --region us-central1 --to-revisions=PREVIOUS=100
# Delete
# gcloud run services delete cyrano-service --region us-central1
# Cost Estimate
# Idle: $0 (scale to zero)
# Active: ~$0.00002400 per second
# Example (1 hour): ~$0.086
# Monitoring
# Console: https://console.cloud.google.com/run/detail/us-central1/cyrano-service/metrics
# Logs: https://console.cloud.google.com/logs/query
# Metrics:
# - Request count
# - Latency
# - Memory usage
# - CPU usage
# Alerts:
# - Error rate > 5%
# - Latency > 500ms
# - Memory > 80%
# Troubleshooting
# Permission denied:
# gcloud auth login
# gcloud auth application-default login
# Secrets not found:
# gcloud secrets list
# gcloud secrets create google-api-key --data-file=-
# Port conflict:
# Change PORT in deploy.sh
# Memory exceeded:
# Increase --memory flag
# Cold start slow:
# Normal: 2-3 seconds
# Consider: --min-instances 1 (costs $)
# Support
# Documentation: https://cloud.google.com/run/docs
# Issues: Check logs first
# Community: Stack Overflow
```

Environment Configuration

```
# .env.example
# Project CYRANO Environment Variables
# Copy to .env and fill in values
# Google Cloud Project
# GOOGLE_CLOUD_PROJECT=gemini-cyrano
# Gemini API Key (from Secret Manager)
# GOOGLE_API_KEY=your-api-key-here
# Session Persistence Backend
# Options: firestore, memory
# SESSION_BACKEND=firestore
# Firestore Collection Name
# FIRESTORE_COLLECTION=cyrano_sessions
# Server Port
# PORT=8080
# Logging Level
# Options: DEBUG, INFO, WARNING, ERROR
# LOG_LEVEL=INFO
# Session Timeout (seconds)
# SESSION_TIMEOUT=1800
# Max Concurrent Connections
# MAX_CONNECTIONS=10
# Development Mode
# Enable hot reload and verbose logging
# DEV_MODE=false
# Cloud Run Service Name
# K_SERVICE=cyrano-service
# Cloud Run Revision
# K_REVISION=cyrano-service-00001-abc
# Cloud Run Region
# GOOGLE_CLOUD_REGION=us-central1
```

Database Setup & Initialization

```
# Firestore Initialization Script
# Project: gemini-cyrano
# Database: (default)
# Location: us-central1
# Prerequisites
# 1. Firestore API enabled
# 2. Authenticated: gcloud auth application-default login
# 3. Project set: gcloud config set project gemini-cyrano
# Enable Firestore API
# gcloud services enable firestore.googleapis.com --project=gemini-cyrano
# Create Firestore Database (if not exists)
# gcloud firestore databases create --location=us-central1 --project=gemini-cyrano
# Verify Database
# gcloud firestore databases list --project=gemini-cyrano
# Expected output:
# NAME: projects/gemini-cyrano/databases/(default)
# LOCATION_ID: us-central1
# TYPE: firestore-native
# Create Collection (optional - auto-created on first write)
# Collection: cyrano_sessions
# Document ID: {session_id}
# Example Document
# gcloud firestore documents create \
# --collection=cyrano_sessions \
# --data-file=- \
# test-session < /dev/stdin
# {
# "session_id": "550e8400-e29b-41d4-a716-446655440000",
# "user_id": "default_user",
# "resumption_handle": "CAESJGNkMjFmZjAtMjAyMy0wMDAwLTAwMDAtMDAwMDAwMDAwMDAwMDAwMDAwMDA",
# "last_updated": 2025-12-31T14:30:45.123Z,
# "created_at": 2025-12-31T14:00:00.000Z,
# "metadata": {
# "client_version": "1.0.0",
# "user_agent": "Mozilla/5.0 (Chrome/120.0.0.0)",
# "ip_address": "hashed_abc123",
# "total_duration_seconds": 1800,
# "turn_count": 45,
# "model_version": "gemini-2.5-flash-native-audio-preview-12-2025"
# }
# }
# Verify Document
# gcloud firestore documents list cyrano_sessions --project=gemini-cyrano
# Expected output:
# NAME: projects/gemini-cyrano/databases/(default)/documents/cyrano_sessions/test-session
# Delete Test Document
# gcloud firestore documents delete cyrano_sessions/test-session --project=gemini-cyrano
# Firestore Security Rules
# rules_version = '2';
# service cloud.firestore {
# match /databases/{database}/documents {
# // Allow service account full access
# match /cyrano_sessions/{session_id} {
# allow read, write: if true;
# }
# }
# }
# Save rules to: firestore.rules
# Deploy rules:
# gcloud firestore rules deploy firestore.rules --project=gemini-cyrano
# Indexes
# Primary: session_id (auto)
# Secondary: user_id + last_updated (composite)
# Create index:
# gcloud firestore indexes composite create \
# --collection=cyrano_sessions \
# --field-config=field-path=user_id,order=ascending \
# --field-config=field-path=last_updated,order=descending \
# --project=gemini-cyrano
# List indexes:
# gcloud firestore indexes composite list --project=gemini-cyrano
# TTL Policy
# Field: last_updated
# Expiration: 1800 seconds (30 minutes)
# Create TTL policy:
# gcloud firestore fields ttl create \
# --collection=cyrano_sessions \
# --field=last_updated \
# --project=gemini-cyrano
# Verify TTL policy:
# gcloud firestore fields ttl list --project=gemini-cyrano
# Backup
# Export:
# gcloud firestore export gs://gemini-cyrano-backup \
# --collection-ids=cyrano_sessions \
# --project=gemini-cyrano
# Import:
# gcloud firestore import gs://gemini-cyrano-backup \
# --collection-ids=cyrano_sessions \
# --project=gemini-cyrano
# Monitoring
# Active sessions:
# gcloud firestore documents count cyrano_sessions --project=gemini-cyrano
# Recent activity:
# gcloud firestore documents list cyrano_sessions --order-by=last_updated --limit=10 --project=gemini-cyrano
# Cleanup
# Manual cleanup (if TTL not working):
# gcloud firestore documents delete --collection=cyrano_sessions --where="last_updated < 2025-12-31T13:00:00" --project=gemini-cyrano
# Troubleshooting
# Permission denied:
# gcloud auth application-default login
# gcloud auth application-default set-quota-project gemini-cyrano
# Database not found:
# gcloud firestore databases create --location=us-central1 --project=gemini-cyrano
# Index required:
# Follow the error message to create the index
# Quota exceeded:
# Check: https://console.cloud.google.com/firestore/quotas
# Request quota increase
# Cost Estimate
# Free tier:
# 1 GiB storage
# 50,000 reads/day
# 20,000 writes/day
# 20,000 deletes/day
# Typical usage:
# Storage: ~1 KB per session
# Reads: 1 per session start
# Writes: 1 per 10 seconds
# Example (1 hour session):
# Writes: 360
# Cost: $0.00
# Support
# Documentation: https://cloud.google.com/firestore/docs
# Console: https://console.cloud.google.com/firestore
# Pricing: https://cloud.google.com/firestore/pricing
```

Deployment Instructions

Prerequisites

- Google Cloud SDK installed
- Authenticated: gcloud auth login
- Project created: gcloud projects create gemini-cyrano
- Billing enabled on the project
- Secrets configured in Secret Manager

Quick Deployment

```
./deploy.sh
```

Manual Deployment

```
gcloud run deploy cyrano-service \
--source . \
--region us-central1 \
--allow-unauthenticated \
--memory 512Mi \
--cpu 1 \
--timeout 300 \
--max-instances 10 \
--set-secrets=GOOGLE_API_KEY=google-api-key:latest
```

Verify Deployment

```
gcloud run services list --region us-central1
gcloud run services describe cyrano-service --region us-central1
```

Monitoring

```
Console: https://console.cloud.google.com/run/detail/us-central1/cyrano-service/metrics
Logs: https://console.cloud.google.com/logs/query
```

Troubleshooting

```
Permission denied: gcloud auth login
Secrets not found: gcloud secrets list
Port conflict: Change PORT in deploy.sh
Memory exceeded: Increase --memory flag
Cold start slow: Normal 2-3 seconds, consider --min-instances 1 (costs $)
```

Support

```
Documentation: https://cloud.google.com/run/docs
Issues: Check logs first
Community: Stack Overflow
```

Success Metrics & Validation

Year 1 Milestones

- Pilot deployment with 3 partner hospitals
- Medical advisory board formed (5 members)
- Clinical validation study initiated
- Regulatory pathway confirmed
- Seed funding secured

3-5 Year Goals

- Multi-site clinical trials (500+ patients)
- Regulatory approval in primary market
- Strategic partnership with medical device company
- Scale to 100 patients/month

Key Indicators

- Clinical outcomes: Pain reduction, functional improvement
- Adoption: Physician referrals, patient satisfaction
- Technical: Accuracy, reliability, safety metrics

Mission Execute: Your Next Actions

Immediate (Next Two Weeks)

- Schedule 5 clinical advisors from your network
- Draft one-page technical specification
- Identify potential pilot hospital contacts
- Research competitive landscape in your region
- Prepare investor deck (10 slides maximum)
- Contact regulatory consultant for pathway assessment

Medium (Next Three Months)

- Complete advisory board formation
- Secure first pilot hospital agreement
- Begin IRB/ethics committee approval
- Prototype refinement based on clinical feedback
- Seed funding conversations
- Partnership discussions with medical device companies

Purpose is Visibility. Keeping You Visible

Mission Complete. Your Next Actions Are Clear. Execute Systematically.